

Marginally Correct Tool Caches Can Reverse Group-Normalized Policy Updates

Shivam Gupta*

Independent Researcher

shivam1720406@gmail.com

20 September 2026

Abstract

Tool-result caching reduces repeated execution in agent training, but reuse also determines which rollouts share environmental randomness. We isolate this statistical choice in a two-action model where independent and shared execution preserve every rollout’s conditional reward distribution. Despite this marginal agreement, sharing one stochastic result per group can reverse the expected group-normalized policy update. We derive an exact finite-group expression: against a constant alternative, the shared update depends on the probability of winning minus the probability of losing, rather than on the difference in expected reward. A Bernoulli specialization gives a wrong-direction region that persists as the group size grows, together with a non-vanishing update-variance floor. Mean centering without group standard-deviation scaling preserves the expected-return direction in this model, an existing remedy rather than a new optimizer. Exhaustive finite sums cover 540 configurations and 3,240 estimator evaluations, with a separately implemented ordered-sequence checker. A separate audit uses 256 scripted rollouts to reproduce the relevant sharing path in a pinned, unmodified TVCache stack. This is a controlled analytical and implementation study, not an LLM training benchmark. It establishes that marginal output validity alone is insufficient to certify a stochastic cache as training-equivalent.

1 Introduction

An execution optimization can change a learner without changing its model, prompt, reward function, or the marginal quality of any one tool response. Consider a training group containing several identical calls to a stochastic tool. Executing each call draws several outcomes. Executing one call and broadcasting its result draws one outcome and repeats it. Both procedures can return the correct distribution to each individual caller, while producing different joint distributions across the group. A learner that compares rollouts within that group can distinguish the two procedures.

This distinction matters for group-relative training. GRPO forms advantages from rewards centered and scaled within a group of completions [1]. Stateful tool caches address a different correctness question: whether reuse respects the environment state that produced an output [6]. State consistency is essential, but does not by itself preserve the sampling law of a stochastic tool. This paper asks a narrower question: *if a cache preserves each rollout’s conditional reward distribution, must it preserve the expected group-normalized update?*

The answer is no. Our counterexample uses an action with a constant payoff and an action with a random payoff. Under shared execution, all occurrences of the random action receive the same newly sampled result within a group. The cache is refreshed for every group. Consequently the effect requires neither stale state nor a permanently unlucky cached answer. The normalizer removes the magnitude of the realized payoff gap, leaving only its sign. For Bernoulli payoffs, the resulting preference threshold is $1/2$, even when the alternative has a substantially higher expected payoff.

Normalization bias is established prior work [2, 3]. We do not introduce it as a new phenomenon, and do not propose removing the standard deviation as a new algorithm. Our contribution is a precisely controlled execution intervention, its exact statistical consequences, and an auditable implementation path. Specifically, we provide:

1. A finite-group characterization of shared-outcome updates for an arbitrary scalar reward distribution, with a Bernoulli sign-reversal example.
2. A separation between marginal reward agreement and update agreement, including asymptotic means, a persistent variance floor, and an existing centered-estimator control.

*LLM-based tools assisted with research exploration, code, mathematical analysis, manuscript preparation, and internal review. Internal automated reviews are not external peer review.

3. Exhaustive numerical checks and a pinned runtime audit that keep mathematical claims, scripted execution evidence, and untested deployment implications separate.

The commercial motivation is practical: a lower physical tool-call count is not, by itself, evidence of equivalent training. The results identify a condition to test when optimizing expensive stochastic tools. They do not measure production prevalence, model accuracy loss, or monetary savings.

2 Related work and scope

Group-relative optimization. DeepSeekMath introduces GRPO as a critic-free group-relative policy optimization method [1]. Liu et al. identify biases associated with group-level standard-deviation scaling and response-length normalization, and propose Dr. GRPO [2]. Our centered-only control uses this existing principle. We analyze a one-step, on-policy score estimator, not the complete clipped, multi-epoch, token-weighted training algorithm.

Reward magnitude and stochastic feedback. Che et al. show that in a sparse-answer regime, group normalization can erase the magnitude of an error penalty and change a decision threshold [3]. Our independent-execution model is affinely equivalent to their three-reward setting. Sharing makes the relevant two-level reward structure arise at every interior action probability, rather than only in a sparse-action regime. This close relationship limits any claim of conceptual novelty. El Mansouri et al. study correction for noisy rewards [4]; Xin measures within-group verifier-error dependence [5]. Here the outcomes need not be erroneous: the intervention changes their dependence directly.

Tool caching. TVCache reuses tool results across stateful trajectories [6]. Its Appendix B assumes that tool outputs depend only on sandbox state and arguments. Our stochastic setting is outside that deterministic-output contract, so the counterexample does not refute its correctness argument. CacheRL studies cached agent-training environments, including cache-aware reward design and masking of injected tool tokens [7]. Correctly excluding environment tokens from the policy loss does not, by itself, establish equivalence of the joint reward law. We neither reproduce those papers’ training results nor claim that their reported benchmarks exhibit our effect.

3 Model and estimators

Let $G \geq 2$ be the rollout-group size. Each rollout independently chooses $a_i \sim \text{Bern}(p)$, where $0 < p < 1$ and $p = \sigma(\theta)$ is parameterized by a logit θ . Action $a_i = 0$, denoted A, returns the constant c . Action $a_i = 1$, denoted B, returns a scalar reward Y with finite first moment $\mu = \mathbb{E}[Y]$. Actions are independent of tool randomness. The expected return and its gradient are

$$J(\theta) = (1 - p)c + p\mu, \quad J'(\theta) = p(1 - p)(\mu - c). \quad (1)$$

The policy score for a sampled action is $\partial_\theta \log \pi_\theta(a_i) = a_i - p$.

Two execution laws. Under *independent execution*, each B invocation receives an independent copy Y_i . Under *shared execution*, one Y is drawn independently for the group and reused for every B invocation. Drawing on the first B cache miss is equivalent to drawing in advance because actions are independent of Y . There is no persistence between groups. In both modes,

$$\mathcal{L}(r_i \mid a_i = 0) = \delta_c, \quad \mathcal{L}(r_i \mid a_i = 1) = \mathcal{L}(Y). \quad (2)$$

Thus the intervention preserves conditional reward marginals, not only their means. When two rollouts choose B, their outcomes are independent in the first mode and identical in the second. This is the only distributional change.

Updates. Write $\bar{r} = G^{-1} \sum_i r_i$ and $s_r^2 = G^{-1} \sum_i (r_i - \bar{r})^2$. We study the ascent estimators

$$U_\epsilon = \frac{1}{G} \sum_{i=1}^G (a_i - p) \frac{r_i - \bar{r}}{s_r + \epsilon}, \quad \epsilon \geq 0, \quad (3)$$

$$V = \frac{1}{G} \sum_{i=1}^G (a_i - p)(r_i - \bar{r}). \quad (4)$$

At $s_r = 0$, U_0 is defined to be zero. Positive updates increase the probability of B. The denominator is the population standard deviation within the group. Using the sample standard deviation changes the scale of U_0 by a positive, group-size-dependent factor, and therefore does not change its sign. We make no such scale-equivalence claim for nonzero ϵ without adjusting that constant.

These estimators describe the reward component of an on-policy update at unit importance ratio. We do not model KL penalties, clipping after parameter movement, adaptive optimizers, variable token lengths, or multi-turn feedback.

4 Exact consequences of sharing

Let $N = \sum_i a_i \sim \text{Bin}(G, p)$ and define

$$w_G(n) = \frac{\sqrt{n(G-n)}}{G}, \quad h_G(n) = w_G(n)^2, \quad S_G(p) = \mathbb{E}[w_G(N)]. \quad (5)$$

For interior p and $G \geq 2$, $S_G(p) > 0$.

Theorem 1 (Shared outcomes replace payoff magnitude by ordering). *Under shared execution, for $\epsilon = 0$,*

$$U_0 = w_G(N) \text{sgn}(Y - c), \quad \mathbb{E}[U_0] = S_G(p)(\Pr(Y > c) - \Pr(Y < c)), \quad (6)$$

where $\text{sgn}(0) = 0$. Consequently, conditional marginal agreement in (2) does not ensure agreement with the expected-return direction.

Proof. Centered rewards sum to zero, so the $-p$ score terms cancel. The centered numerator equals $h_G(N)(Y - c)$. The shared group contains only the two reward values c and Y , giving $s_r = w_G(N)|Y - c|$. Division yields (6) when $0 < N < G$ and $Y \neq c$. Both sides are zero otherwise. Finally N and Y are independent, so their expectations factor. $\square$

The theorem concerns the *direction of the update*, not a claim that an arbitrary full learning algorithm maximizes a median objective. It separates the frequency of beating an alternative from the value of doing so. A small gain on many calls and a large loss on fewer calls can have positive ordering balance but negative mean advantage.

Corollary 2 (Bernoulli threshold and wrong-direction region). *For $Y \sim \text{Bern}(q)$ and $0 < c < 1$,*

$$\mathbb{E}[U_{0,\text{share}}] = (2q - 1)S_G(p). \quad (7)$$

For every $G \geq 2$ and interior p , the shared normalized update has the opposite sign to J' whenever $1/2 < q < c < 1$ or $0 < c < q < 1/2$.

The threshold is $q = 1/2$ for the shared update and $q = c$ for expected return. This statement does not say that independent normalization is unbiased. At $G = 2$, both modes have the same expected update: a group containing both actions includes only one stochastic B result.

Proposition 3 (Centering preserves the mean-gradient direction). *For either execution law and any integrable Y ,*

$$\mathbb{E}[V] = \left(1 - \frac{1}{G}\right)p(1-p)(\mu - c). \quad (8)$$

Multiplication by $G/(G-1)$ removes the self-including-baseline factor.

Proof. Conditional on $N = n$, the centered numerator is $(G-n)(\sum_{i:a_i=1} r_i - nc)/G^2$. Both laws give conditional sum expectation $n\mu$. Apply $\mathbb{E}[N(G-N)] = G(G-1)p(1-p)$. $\square$

This is an existing baseline correction. It does not restore the entire distribution of the update, nor establish equivalence after nonlinear clipping or adaptive optimizer transformations.

Theorem 4 (A persistent mean shift and variance floor). *Suppose $Y \sim \text{Bern}(q)$ with $0 < q < 1$, $0 < c < 1$, and fixed interior p . As $G \rightarrow \infty$,*

$$\mathbb{E}[U_{0,\text{ind}}] \rightarrow \frac{p(1-p)(q-c)}{\sqrt{pq(1-q) + p(1-p)(q-c)^2}}, \quad (9)$$

$$\mathbb{E}[U_{0,\text{share}}] \rightarrow (2q-1)\sqrt{p(1-p)}, \quad (10)$$

$$\text{Var}(U_{0,\text{ind}}) \rightarrow 0, \quad \text{Var}(U_{0,\text{share}}) \rightarrow 4q(1-q)p(1-p). \quad (11)$$

At finite G , the shared variance is exactly

$$\text{Var}(U_{0,\text{share}}) = (1 - 1/G)p(1-p) - (2q-1)^2 S_G(p)^2. \quad (12)$$

Proof. Under independent execution, laws of large numbers give the covariance $p(1-p)(q-c)$ and reward variance $pq(1-q) + p(1-p)(q-c)^2$. Under sharing, use (6) and $N/G \rightarrow p$. In either mode, Cauchy–Schwarz gives $|U_0| \leq \sqrt{(N/G)(1-N/G)} \leq 1/2$. Bounded convergence therefore applies to first and second moments. For the shared second moment, $(Y-c)$ is never zero, so $U_0^2 = h_G(N)$ and $\mathbb{E}[U_0^2] = (1 - 1/G)p(1-p)$. Subtracting the squared mean proves (12) and its limit. $\square$

In the wrong-direction regions, sufficiently large independent groups and shared groups point in opposite directions. Increasing the number of policy samples does not increase the number of independent tool outcomes in the shared group. Independent groups can still average down this noise; the result is a within-group variance floor, not irreducible uncertainty across all data.

4.1 Nonzero numerical stabilization

For $\epsilon > 0$ in the Bernoulli setting, define

$$A_\epsilon = \mathbb{E} \left[\frac{h_G(N)(1-c)}{w_G(N)(1-c) + \epsilon} \right], \quad B_\epsilon = \mathbb{E} \left[\frac{h_G(N)c}{w_G(N)c + \epsilon} \right]. \quad (13)$$

Then $\mathbb{E}[U_{\epsilon,\text{share}}] = qA_\epsilon - (1-q)B_\epsilon$ and the zero-update threshold is $q_\epsilon = B_\epsilon / (A_\epsilon + B_\epsilon)$. For $c > 1/2$, $1/2 < q_\epsilon < c$; for $c < 1/2$, $c < q_\epsilon < 1/2$. These inequalities follow termwise by comparing A_ϵ and B_ϵ , and $(1-c)B_\epsilon$ with cA_ϵ . As $\epsilon \downarrow 0$ the threshold tends to $1/2$; as $\epsilon \rightarrow \infty$ it tends to c . A nonzero stabilizer therefore does not automatically restore the intended preference threshold. Appendix C states the endpoint conventions.

5 Exhaustive numerical verification

5.1 Protocol and controls

Before executing the sweep, we fixed $G \in \{2, 4, 8, 16, 32, 64\}$, $p \in \{0.1, 0.5, 0.9\}$, $c \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, and $q \in \{0, 0.2, 0.4, 0.6, 0.8, 1\}$. This gives 540 configurations. For each configuration we evaluate both execution modes with centering alone and with normalization at $\epsilon \in \{0, 10^{-4}\}$, for 3,240 estimator evaluations. These are finite probability-weighted sums computed in floating arithmetic, not Monte Carlo observations or independent trials.

With $N = n$, let K count successful B outcomes. Independent execution uses $K \mid N = n \sim \text{Bin}(n, q)$; shared execution uses $K \in \{0, n\}$ with probabilities $1 - q, q$. The centered numerator is

$$V(n, k) = \frac{(G-n)(k-nc)}{G^2}. \quad (14)$$

The implementation computes the reward variance from centered squared deviations, avoiding subtraction of nearly equal second moments. It accumulates weighted sums with compensated summation.

A separately implemented checker enumerates ordered rollout outcomes and evaluates the literal score expression, rather than the count reduction. It checks 288 small-group combinations. Other controls verify probability mass, the reward mean, the centered identity, the shared formula, deterministic $q \in \{0, 1\}$ equivalence, and $G = 2$ equivalence. All six test methods pass. Maximum probability-mass error is below 3.34×10^{-15} ; maximum marginal-mean error is below 1.78×10^{-15} . Numerical precision checks are not statistical confidence intervals.

5.2 A preselected witness

The protocol selected $p = 0.5$, $c = 0.9$, $q = 0.8$ analytically before execution. Both laws have mean reward 0.85 and true gradient -0.025 . At $G = 64$, independent normalization gives -0.078867 , while sharing gives $+0.297628$. The centered estimator gives -0.024609 under either law (Table 1). Thus the intervention reverses the expected update while leaving the conditional reward marginals unchanged.

Table 1: Exact finite-sum witness, rounded to six decimals. Positive updates increase B, the lower-mean action. Normalized columns use $\epsilon = 0$.

G	Independent normalized	Shared normalized	Centered, either law
2	0.150000	0.150000	-0.012500
4	0.187129	0.242404	-0.018750
8	0.111526	0.278348	-0.021875
16	0.005998	0.290118	-0.023438
32	-0.059543	0.295196	-0.024219
64	-0.078867	0.297628	-0.024609

Sharing stochastic tool results can reverse the update

Constructed bandit: A always returns 0.9; B succeeds with probability 0.8. Initial $P(B) = 0.5$.

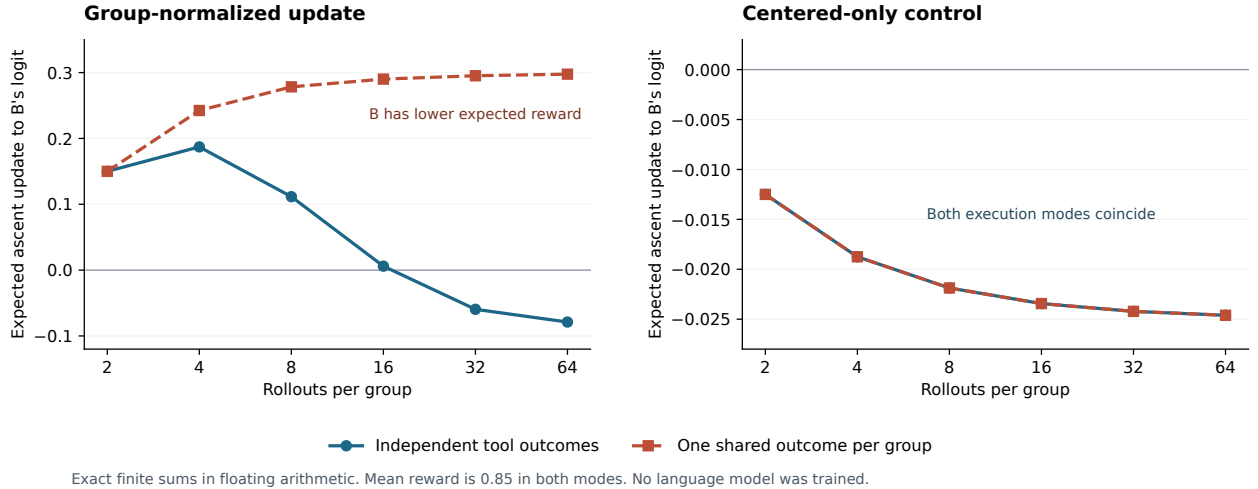


Figure 1: Increasing rollout count separates the two normalized updates in the preselected witness. Centering alone preserves their expected direction. Panels have different vertical scales. Curves are exhaustive calculations, not learning curves or measured model performance.

Across the entire specified grid, the modes have opposite normalized-update signs in 54 of 540 configurations at each tested ϵ . Independent normalization opposes the expected-return gradient in 54 configurations, and shared normalization in 108. Centering alone has no such sign discrepancy. The deterministic controls agree exactly. The grid was constructed to examine the mechanism; its fractions must not be interpreted as deployment prevalence. The $\epsilon = 10^{-4}$ result verifies robustness to one practical numerical stabilizer, not all implementation conventions.

6 A pinned implementation audit

6.1 Execution boundary

To connect the intervention to executable infrastructure, we inspected and ran TVCache revision 3a4f95a6582e [8]. The adaptive follow-up protocol was written after source inspection and after the numerical study, before executing the runtime probe. The probe uses unmodified implementations of the semantic stateful executor, async client, Flask routes, and

immutable prefix-tree backend. Only the HTTP transport is replaced with an in-process bridge to Flask’s test client. A scripted environment implements the tool interface. Socket connections and DNS resolution are disabled during execution.

Each case has 32 sequential one-call rollouts, alternating A and B, with a new executor for each rollout and a shared backend within the case. A returns the string 0.9. B either always returns 0.8, or alternates 1, 0 on physical execution, starting with 1. The latter is a deliberately varying stream, not a Bernoulli sample. We compare direct execution, shared identity, a separate task namespace per rollout, and a separate draw identifier in tool arguments.

Table 2: Runtime audit. Counts are per case of 32 scripted rollouts. Each mode is run once with constant outputs and once with a varying stream.

Mode	Physical calls	Returned B stream in varying fixture
Direct execution	32	Alternating 1, 0
Shared identity	2	Sixteen copies of 1
Separate task namespace	32	Alternating 1, 0
Separate draw identity	32	Alternating 1, 0

6.2 Observed behavior and interpretation

All eight cases satisfy the protocol’s expectations (Table 2). The 256 scripted rollouts include 64 direct-environment controls and 192 rollouts through the cache stack. In total there are 196 physical tool calls and 648 in-process HTTP requests. The socket and DNS guards record zero attempts. Three full runs in separate processes produce byte-identical result JSON. Before import, all 29 exported source files pass hash verification.

The shared case executes only one A call and one B call. Reuse preserves the constant-output control and repeats the first B result in the varying fixture. Identity-separated controls retain all physical executions. These controls trade away reuse; they are not a new or cost-free fix. The experiment establishes an implementation path for sharing, not a failure of the vendor’s stated deterministic contract or a measured training regression.

The inspected video integration labels loading and preprocessing as state-changing and includes a model-backed caption call. This motivates a separate variability measurement, which we have not performed. Importantly, TVCache’s paper lists its EgoSchema training objective as importance sampling, while other workloads use GRPO [6]. We therefore do not infer that its caption workload uses the group-normalized estimator studied here.

7 Implications, limitations, and reproducibility

Cache equivalence is estimator-dependent. For the mean-centered estimator in this model, sharing preserves the expected gradient up to the usual finite-group factor. For the normalized estimator, it need not. An appropriate training audit must specify both the execution law and the consuming estimator. Matching tool-response means or individual output distributions alone is weaker than matching the expected update.

Cost must be reported separately. In the one-step Bernoulli model, independent execution requires Gp expected physical B calls per group, whereas sharing requires $1 - (1 - p)^G$. That saving comes with different gradient variance. Equal-rollout comparisons and equal-physical-call comparisons answer different questions. Our work reports no time-to-accuracy or dollar-efficiency result. Appendix B gives exact centered-estimator variances to support such comparisons without assuming that identical expected gradients imply identical learning behavior.

Scope of the evidence. There are two actions, one decision per rollout, a fixed policy, and a constant alternative. Theorems concern a specific on-policy reward estimator. We do not train or evaluate a language model, establish an end-to-end convergence result, or measure the frequency of stochastic tools in deployed agents. Actual multi-turn policies can respond to cached observations, changing their action distribution as well as their rewards. Cache lifetime, concurrent misses, partial sharing, eviction, and stale state can introduce additional effects outside the model. A production result would require representative tool distributions, measured reuse, repeated training runs, and equal-budget comparisons with existing optimizers. Automated internal review does not replace independent scientific review or the author’s responsibility for the manuscript.

Reproducibility. The artifact contains pre-execution protocols, the independent checker, all finite-sum results, runtime traces, source and result hashes, exact dependency versions, and the figure-generation code. The numerical study requires only Python’s standard library; plotting uses Matplotlib. The runtime probe uses Python 3.12 with Flask 3.1.2, HTTPX 0.28.1, and Requests 2.32.5, with all transitive versions pinned. Third-party code is retrieved from the pinned Git revision, kept unmodified, and excluded from redistribution. No credentials, paid API calls, participant data, or model outputs are required to reproduce the reported experiments. The public artifact, including source and numerical results, is available at <https://github.com/shi1720/tool-cache-coupling>.

Broader impact. The intended use is auditing training optimizations before relying on their efficiency claims. A marginally valid cache can be useful and still change the learning problem. Conversely, this constructed counterexample should not be used to assert that tool caching is generally unsafe or that a named system’s published results are invalid.

8 Conclusion

Sharing a stochastic tool result changes the joint sampling law seen by a group-relative learner. In a controlled model with identical conditional reward marginals, the shared normalized update follows an ordering comparison rather than the mean-payoff advantage. Sharing can reverse the expected normalized update and leave a within-group variance floor as rollout count grows. Exhaustive calculations and a pinned runtime audit make the mechanism reproducible. The practical lesson is specific: evaluate stochastic reuse against the estimator it serves, and measure physical tool outcomes separately from rollout count.

References

- [1] Zhihong Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024. <https://arxiv.org/abs/2402.03300>.
- [2] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv preprint arXiv:2503.20783*, 2025. <https://arxiv.org/abs/2503.20783>.
- [3] Xujun Che, Yuchen Yuan, Weida Zhao, and Chenyang Yu. Abstention as an Action Can Kill Both the Reward Gradient and the KL Anchor: Collapse Law and Repair for Error-Penalized Reinforcement Learning. *arXiv preprint arXiv:2608.00301*, 2026. <https://arxiv.org/abs/2608.00301>.
- [4] Omar El Mansouri, Fathinah Asma Izzati, Mohamed El Amine Seddik, and Salem Lahlou. Noise-corrected GRPO: From Noisy Rewards to Unbiased Gradients. *arXiv preprint arXiv:2510.18924*, 2025. <https://arxiv.org/abs/2510.18924>.
- [5] Esther Xin. Are Verifier Errors Independent Within a GRPO Group? Evidence from Qwen2.5 Rollouts. *arXiv preprint arXiv:2609.06386*, 2026. <https://arxiv.org/abs/2609.06386>.
- [6] Abhishek Vijaya Kumar, Bhaskar Kataria, Byungsoo Oh, Emaad Manzoor, and Rachee Singh. TVCache: A Stateful Tool-Value Cache for Post-Training LLM Agents. *arXiv preprint arXiv:2602.10986*, 2026. <https://arxiv.org/abs/2602.10986>.
- [7] Md Amirul Islam, Sumiran Thakur, Huancheng Chen, Su Min Park, Jiayun Wang, and Gyuhak Kim. CacheRL: Multi-Turn Tool-Calling Agents via Cached Rollouts and Hybrid Reward. *arXiv preprint arXiv:2606.14179*, 2026. <https://arxiv.org/abs/2606.14179>.
- [8] TVCache authors. TVCache source repository, revision 3a4f95a6582eea1e4b7e84a9f3a0c74eaa8fde02. Accessed 20 September 2026. <https://github.com/TVCache/TVCache>.

A Exact summation and equality controls

For $b_G(n; p) = \binom{G}{n} p^n (1-p)^{G-n}$, the independent expected update is

$$\mathbb{E}[U_{\epsilon, \text{ind}}] = \sum_{n=0}^G b_G(n; p) \sum_{k=0}^n \binom{n}{k} q^k (1-q)^{n-k} \frac{(G-n)(k-nc)}{G^2(s(n, k) + \epsilon)}, \quad (15)$$

where the fraction is zero for a zero denominator and

$$m(n, k) = ((G-n)c + k)/G, \quad (16)$$

$$s(n, k)^2 = [(G-n)(c-m)^2 + k(1-m)^2 + (n-k)m^2]/G. \quad (17)$$

The shared sum replaces the inner binomial distribution with mass $1-q$ at $k=0$ and mass q at $k=n$. When $n=0$, only one outcome is counted, with total mass one. The numerical implementation stores the update expectation and variance, probability mass, and expected reward for every estimator.

The deterministic controls $q=0$ and $q=1$ give the same reward vector under both laws, conditional on the action vector. Their update distributions agree, not just their means. For $G=2$, groups with equal actions have identical scores and a zero centered update even if their rewards differ. Mixed-action groups have one B sample, so the update distributions agree in that case too. These facts explain the zero differences observed by the controls.

B Variance of the centered estimator

In the Bernoulli model, condition on N and write $h = h_G(N)$. The law of total variance gives

$$\text{Var}(V_{\text{share}}) = q(1-q)\mathbb{E}[h^2] + (q-c)^2 \text{Var}(h), \quad (18)$$

$$\text{Var}(V_{\text{ind}}) = q(1-q)\mathbb{E}\left[\frac{N(G-N)^2}{G^4}\right] + (q-c)^2 \text{Var}(h). \quad (19)$$

The conditional means are the same, $h(q-c)$. Sharing replaces the conditional variance of a sum of N independent Bernoulli outcomes, $Nq(1-q)$, by $N^2q(1-q)$. Substitution into the squared coefficient $(G-N)^2/G^4$ proves the formulas. Correcting the baseline factor multiplies either variance by $G^2/(G-1)^2$.

These expressions make an important distinction explicit: a mean-direction repair does not preserve gradient noise. Any efficiency claim must account for the number of independently sampled groups and the physical call budget. We do not turn these identities into a wall-clock or convergence claim.

C Stabilizer threshold and general reward ties

For $\epsilon > 0$, A_ϵ and B_ϵ are strictly positive at interior p and c . If $c > 1/2$, then for every $0 < n < G$, the corresponding summands satisfy $B_n > A_n$ and $(1 - c)B_n < cA_n$. The first follows because $x/(wx + \epsilon)$ is increasing in positive x ; the second follows by comparing the denominators after the common numerator $hc(1 - c)$ is factored out. Summing preserves both strict inequalities, yielding $1/2 < q_\epsilon < c$. The reversed inequalities apply when $c < 1/2$; when $c = 1/2$, the threshold is exactly $1/2$. Boundary action counts contribute zero. Finite sums permit both limits in ϵ to be taken termwise.

For general shared Y , ties at $Y = c$ contribute zero. The exact second moment is $\mathbb{E}[U_0^2] = (1 - 1/G)p(1 - p) \Pr(Y \neq c)$. Thus the variance extension of Theorem 4 subtracts $S_G(p)^2 [\Pr(Y > c) - \Pr(Y < c)]^2$ from this expression. Finite first moment is needed to compare with expected return; the bounded normalized update itself is well-defined for any almost surely finite scalar Y .

D Artifact map and audit boundaries

Artifact	Purpose
<code>docs/protocol-001.md</code>	Fixed grid and controls before numerical execution.
<code>src/enumerate_updates.py</code>	Count-based finite-sum computation.
<code>tests/test_enumeration.py</code>	Independent ordered-outcome checks and identities.
<code>reviews/check_extensions.py</code>	Variance and stabilizer identities checked against frozen results.
<code>results/enumeration-001.json</code>	All 540 configurations, including negative controls.
<code>docs/protocol-002.md</code>	Explicitly adaptive runtime follow-up.
<code>src/runtime_probe.py</code>	Unmodified vendor stack with in-process transport.
<code>results/runtime-002.json</code>	Full returned sequences and execution counters.
<code>results/*manifest*.json</code>	Source and result digests for reproducibility.
<code>requirements-runtime.lock</code>	Exact runtime dependency versions.

The runtime audit tests a narrow execution path: sequential one-call rollouts, no stored sandbox forks, no eviction, no expiry, and no production network. The backend’s cleanup threads are stopped after each case. Distinct identities are supplied by the experiment, not discovered by an agent. The varying-output fixture is not the probabilistic environment used in the theorem, and its returned means must not be used as evidence for marginal agreement. That agreement is proved and numerically verified only in the analytical study.